Добавь сценарное прогнозирование и оптимизацию

Добавим два блока: **сценарное прогнозирование** (что будет при разных параметрах) и **оптимизацию** (какие параметры дают лучший результат). Это превратит модель из «описательной» в **инструмент управления**.

Что добавим

- Сценарный анализ: базовый, пессимистичный, оптимистичный, кризисный.
- Оптимизация: подбор параметров, которые максимизируют $Q(t)$ и не допускают разрушения контуров.
- Сохранение результатов сценариев и оптимизации в CSV.
- Визуализация сценариев и оптимальной траектории.

Обновлённый код

```
import os
import sys
import subprocess
import importlib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib import animation
from scipy.integrate import solve_ivp
from scipy.optimize import differential_evolution
from dataclasses import dataclass, field
from typing import List, Tuple
from fpdf import FPDF

# ------------------------------
# Проверка и установка библиотек
# ------------------------------

REQUIRED = ['numpy', 'scipy', 'matplotlib', 'pandas', 'fpdf']

def ensure_dependencies():
    for pkg in REQUIRED:
        try:
            importlib.import_module(pkg)
        except ImportError:
            print(f"Устанавливаю {pkg}...")
            subprocess.check_call([sys.executable, '-m', 'pip', 'install', pkg])
```

```
ensure_dependencies()

# ------------------------------
# Параметры модели
# ------------------------------

@dataclass
class ModelParams:
    lambda_: np.ndarray = field(default_factory=lambda: np.array([0.5]*6))
    alpha: np.ndarray = field(default_factory=lambda: np.array([0.2]*6))
    beta: np.ndarray = field(default_factory=lambda: np.array([0.1]*6))
    gamma: np.ndarray = field(default_factory=lambda: np.array([0.1]*6))
    delta: np.ndarray = field(default_factory=lambda: np.array([0.05]*6))
    theta: np.ndarray = field(default_factory=lambda: np.array([0.3]*6))
    tau_E: float = 2.0

# ------------------------------
# Ядро модели
# ------------------------------

class LivingEquilibriumModel:
    def __init__(self, params: ModelParams):
        self.p = params

    def errors(self, x):
        return np.maximum(0, self.p.theta - x)

    def derivative(self, t, x):
        S, O, M, P, A, D = x
        lam, alpha, beta, gamma, delta, theta = (
            self.p.lambda_, self.p.alpha, self.p.beta,
            self.p.gamma, self.p.delta, self.p.theta
        )
        e = self.errors(x)
        x_bar = np.mean(x[:5])

        dS = -lam[^0]*S + alpha[^0]*e[^0] + beta[^0]*O*M - gamma[^0]*P - delta[^0]*(1
        dO = -lam[^1]*O + alpha[^1]*e[^1] + beta[^1]*S*M - gamma[^1]*P - delta[^1]*(1
        dM = -lam[^2]*M + alpha[^2]*e[^2] + beta[^2]*S*O - gamma[^2]*P - delta[^2]*(1
        dP = -lam[^3]*P + alpha[^3]*e[^3] + beta[^3]*S*O*M - gamma[^3]*A - delta[^3]*
        dA = -lam[^4]*A + alpha[^4]*e[^4] + beta[^4]*P - gamma[^4]*(S+O+M) + delta[^4
        dD = -lam[^5]*D + alpha[^5]*e[^5] + beta[^5]*(S+O+M) - gamma[^5]*x_bar

        return [dS, dO, dM, dP, dA, dD]

# ------------------------------
# Запуск симуляции
# ------------------------------

def run_simulation(params: ModelParams, x0=None, t_span=(0, 20), n_points=500):
    if x0 is None:
        x0 = np.array([0.8, 0.8, 0.8, 0.8, 0.8, 0.8])
    model = LivingEquilibriumModel(params)
    t_eval = np.linspace(*t_span, n_points)
    sol = solve_ivp(model.derivative, t_span, x0, t_eval=t_eval, method='RK45')
```

```
    return sol

# ----------------------------
# Сценарное прогнозирование
# ----------------------------

def scenario_analysis():
    scenarios = {
        'base': ModelParams(),
        'pessimistic': ModelParams(
            lambda_=np.array([0.3]*6),
            alpha=np.array([0.1]*6),
            gamma=np.array([0.2]*6),
            delta=np.array([0.1]*6),
        ),
        'optimistic': ModelParams(
            lambda_=np.array([0.7]*6),
            alpha=np.array([0.3]*6),
            gamma=np.array([0.05]*6),
            delta=np.array([0.02]*6),
        ),
        'crisis': ModelParams(
            lambda_=np.array([0.1]*6),
            alpha=np.array([0.05]*6),
            gamma=np.array([0.3]*6),
            delta=np.array([0.2]*6),
        ),
    }

    results = {}
    for name, params in scenarios.items():
        sol = run_simulation(params)
        results[name] = sol
    return results

def save_scenarios_to_csv(results, filename='output/scenarios.csv'):
    all_data = []
    for name, sol in results.items():
        df = pd.DataFrame({
            'scenario': name,
            'time': sol.t,
            'S': sol.y[^0],
            'O': sol.y[^1],
            'M': sol.y[^2],
            'P': sol.y[^3],
            'A': sol.y[^4],
            'D': sol.y[^5],
        })
        all_data.append(df)
    pd.concat(all_data).to_csv(filename, index=False)
    print(f"Сценарии сохранены в {filename}")

# ----------------------------
# Оптимизация
# ----------------------------
```

```
def objective(params_vector):
    # Распаковываем параметры
    lam = params_vector[:6]
    alpha = params_vector[6:12]
    gamma = params_vector[12:18]
    delta = params_vector[18:24]

    params = ModelParams(
        lambda_=lam,
        alpha=alpha,
        gamma=gamma,
        delta=delta,
    )

    sol = run_simulation(params)
    if not sol.success:
        return 1e6

    # Целевая функция: максимизировать среднее Q и не допустить нулей
    Q = np.mean(sol.y, axis=1)
    penalty = np.sum(np.clip(0.1 - sol.y, 0, None)**2)
    return -np.mean(Q) + 1000*penalty

def optimize_parameters():
    bounds = [(0.1, 1.0)]*6 + [(0.05, 0.5)]*6 + [(0.01, 0.3)]*6 + [(0.01, 0.2)]*6
    result = differential_evolution(
        objective,
        bounds,
        maxiter=20,
        popsize=10,
        tol=1e-3,
        seed=42
    )
    return result

# ------------------------------
# Визуализация
# ------------------------------

def plot_scenarios(results):
    labels = ['S', 'O', 'M', 'P', 'A', 'D']
    plt.figure(figsize=(12, 6))
    for name, sol in results.items():
        plt.plot(sol.t, sol.y[^0], label=f'{name} (S)')
    plt.xlabel('Время')
    plt.ylabel('S (выживание)')
    plt.title('Сценарный анализ: выживание')
    plt.legend()
    plt.grid(True)
    plt.savefig('output/scenarios_S.png')
    plt.close()

def plot_optimal(sol):
    labels = ['S', 'O', 'M', 'P', 'A', 'D']
    plt.figure(figsize=(12, 6))
    for i in range(6):
```

```
        plt.plot(sol.t, sol.y[i], label=labels[i])
    plt.xlabel('Время')
    plt.ylabel('Значение')
    plt.title('Оптимальная траектория')
    plt.legend()
    plt.grid(True)
    plt.savefig('output/optimal_trajectory.png')
    plt.close()

# ------------------------------
# Главная функция
# ------------------------------

def main():
    os.makedirs('output', exist_ok=True)

    # Базовая симуляция
    sol = run_simulation(ModelParams())
    df = pd.DataFrame({
        'time': sol.t,
        'S': sol.y[^0],
        'O': sol.y[^1],
        'M': sol.y[^2],
        'P': sol.y[^3],
        'A': sol.y[^4],
        'D': sol.y[^5],
    })
    df.to_csv('output/model_results.csv', index=False)

    # Сценарный анализ
    scenarios = scenario_analysis()
    save_scenarios_to_csv(scenarios, 'output/scenarios.csv')
    plot_scenarios(scenarios)

    # Оптимизация
    result = optimize_parameters()
    print("Оптимальные параметры:", result.x)
    print("Значение целевой функции:", result.fun)

    # Симуляция с оптимальными параметрами
    lam = result.x[:6]
    alpha = result.x[6:12]
    gamma = result.x[12:18]
    delta = result.x[18:24]
    optimal_params = ModelParams(lambda_=lam, alpha=alpha, gamma=gamma, delta=delta)
    optimal_sol = run_simulation(optimal_params)
    plot_optimal(optimal_sol)

    # Визуализация базовой модели
    plot_2d(sol)
    plot_3d(sol)
    animate(sol)

if __name__ == "__main__":
    main()
```

Что добавлено

- **Сценарный анализ**: 4 сценария (базовый, пессимистичный, оптимистичный, кризисный).
- **Оптимизация**: подбор параметров через `differential_evolution`.
- **Сохранение сценариев** в `scenarios.csv`.
- **Визуализация сценариев** и оптимальной траектории.

Что делает оптимизация

- Ищет параметры, которые **максимизируют среднее значение контуров**.
- Штрафует за приближение к нулю любого контура.
- Использует глобальный поиск (`differential_evolution`), чтобы не застрять в локальном минимуме. [1] [2] [3]

Что появится в `output/`

- `model_results.csv` — базовая симуляция.
- `scenarios.csv` — все сценарии.
- `scenarios_S.png` — график выживания по сценариям.
- `optimal_trajectory.png` — оптимальная траектория.
- `model_dynamics_2d.png` — 2D-график.
- `phase_portrait_3d.png` — 3D-график.

Финальная формулировка

> **Сценарии показывают, что может случиться. Оптимизация показывает, что можно сделать.**

Это и есть переход от «описательной модели» к **инструменту управления**: не просто «что будет», а **«как сделать лучше»**. [2] [4] [5] [1]

⁂

1. https://docs.scipy.org/doc//scipy-1.16.0/tutorial/optimize.html
2. https://docs.scipy.org/doc/scipy/reference/generated//scipy.optimize.differential_evolution.html
3. https://docs.scipy.org/doc/scipy-1.16.2/reference/generated/scipy.optimize.differential_evolution.html
4. https://arxiv.org/pdf/2607.15470.pdf
5. https://pythonhealthdatascience.github.io/des_rap_book/pages/guide/experiments/scenarios.html
6. https://en.wikipedia.org/wiki/On_Liberty
7. https://docs.scipy.org/doc//scipy-0.15.0/reference/tutorial/optimize.html
8. https://docs.scipy.org/doc//scipy-1.9.0/reference/generated/scipy.optimize.differential_evolution.html
9. https://docs.scipy.org/doc/scipy/tutorial/optimize.html

10. https://docs.scipy.org/doc/scipy-1.14.0/reference/generated/scipy.optimize.differential_evolution.html
11. https://docs.scipy.org/doc/scipy-1.11.0/reference/generated/scipy.optimize.differential_evolution.html
12. https://docs.gurobi.com/projects/examples/en/14.0/examples/python/sensitivity.html
13. https://docs.scipy.org/doc/scipy-1.14.1/reference/generated/scipy.optimize.differential_evolution.html
14. https://docs.gurobi.com/projects/examples/en/14.0/examples/sensitivity.html
15. https://docs.gurobi.com/projects/examples/en/14.0/examples/cs/sensitivity_cs.html
16. https://lectures.scientific-python.org/_sources/advanced/mathematical_optimization/index.md
17. https://www.britannica.com/print/article/624300
18. https://www.annenbergclassroom.org/glossary_term/majority-rule-and-minority-rights/
19. https://www.britannica.com/topic/veche
20. https://www.victorianweb.org/victorian/philosophy/mill/liberty.html